



American International University - Bangladesh

Department of Electrical and Electronic Engineering

EEE 4103: Microprocessor and Embedded Systems Laboratory

Title: Familiarization with a microcontroller, the study of the blink test, and implementation of a traffic control system using microcontrollers.

Introduction:

The objective of this experiment is to get familiar with Microcontrollers. Besides, the specific objectives of this experiment are to

- ✓ Make an LED blink using an Arduino and its delay function.
- ✓ Implement a traffic control system using Arduino and its delay function and LEDs.

Theory and Methodology:

Arduino is an open-source platform used for creating interactive electronics projects. Arduino consists of both a programmable microcontroller and a piece of software, or Integrated Development Environment (IDE), that runs on your computer, used to write and upload computer code to the microcontroller board. Arduino Uno also doesn't need a hardware circuit (programmer/burner) to load a new code into the board. We can easily load code into the board just using a Universal Serial Bus (USB) cable and the Arduino IDE (which uses an easier version of C++ to write code).

Overview of Arduino Board :

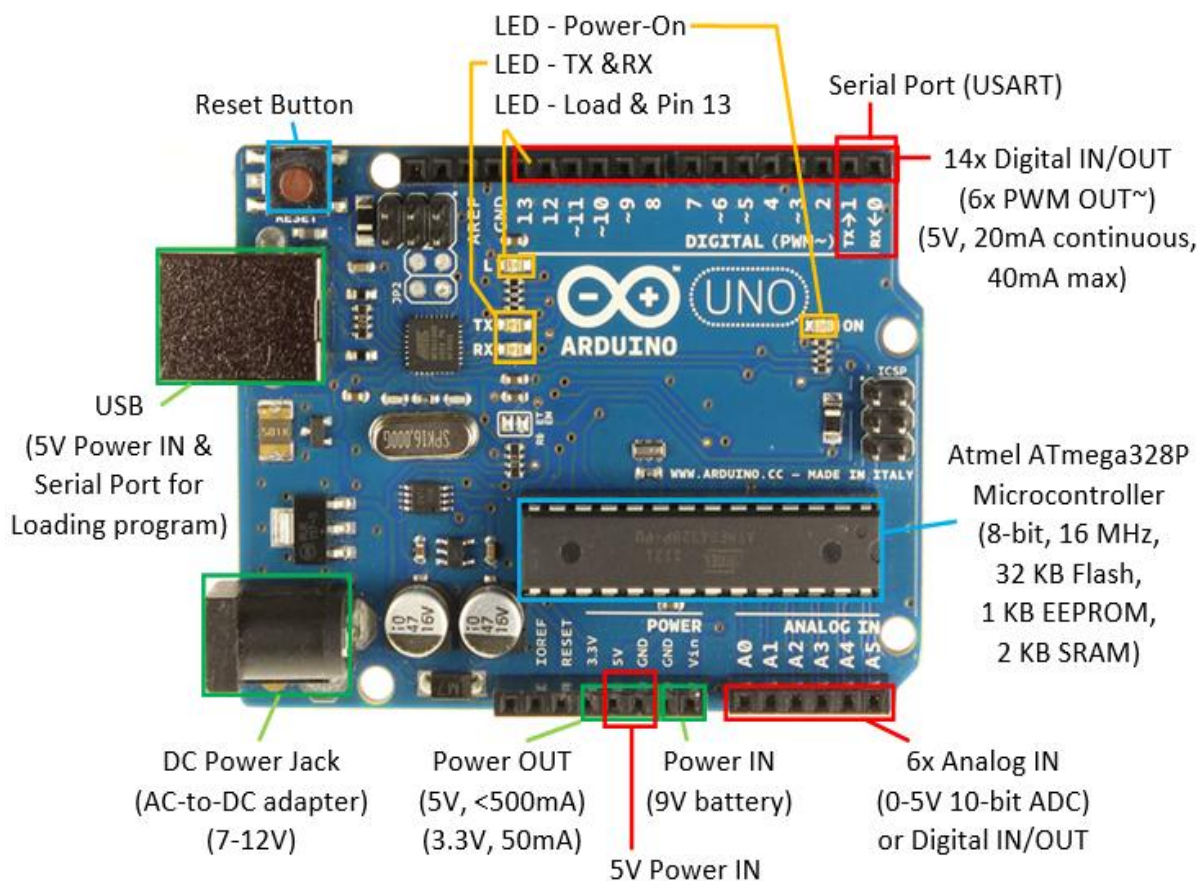


Fig. 1 Arduino Board

Apparatus:

- 1) Arduino IDE (any version)
- 2) Arduino Uno (R3) board or Arduino mega 2560 board
- 3) LED lights (Red, Green, and Yellow)
- 4) Three 100 ohms resistors, and
- 5) Jumper wires

Using Arduino IDE to write code:

1. Open the Arduino Uno IDE 1.8.2 (version may have been updated automatically on your computer) and a blank sketch will open. The following window will come up on your PC:

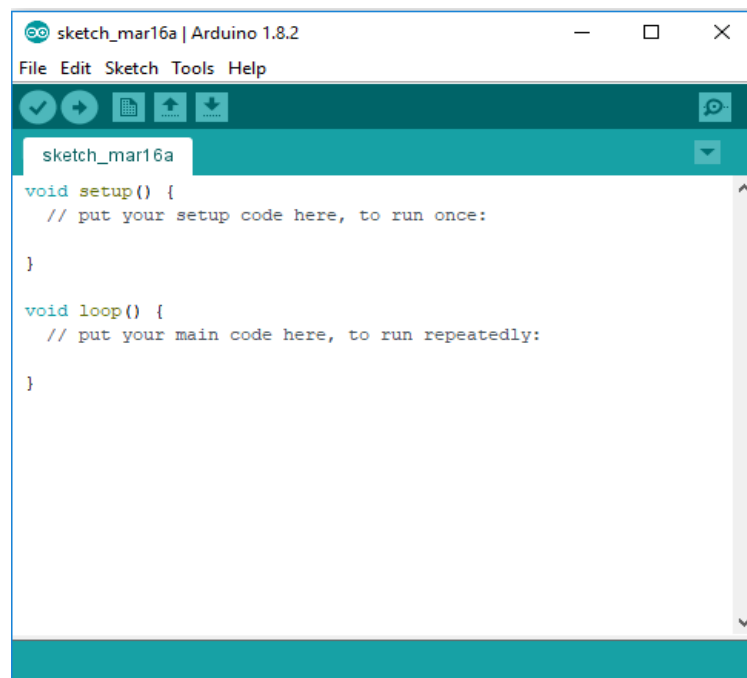


Fig. 2 IDE Environment (text editor)

2. Now, write the following program on the blank sketch for the LED blink test.

```
void setup() {  
  // Pin type declaration for the red LED  
  pinMode(5, OUTPUT);  
}  
  
void loop() {  
  // Turning on the voltage at output pin 5 (for red LED)  
  
  digitalWrite(5, HIGH);  
  delay(1000); // LED is turned on for 1 second  
  digitalWrite(5, LOW);  
  delay(1000); // LED is turned off for 1 second  
}
```

Experimental Procedure:

The main task of our lab is to understand and implement a traffic light control system after learning how to blink an LED. First, make the circuits using the connection diagrams provided. The first diagram (a) is for the LED blink test, and the second one (b) is for the traffic light control system experiments. Please note that the LED has two terminals: an anode and a cathode. The anode leg is slightly longer than the cathode leg. The anode must be connected to the output pin, and the cathode must be connected to the ground.

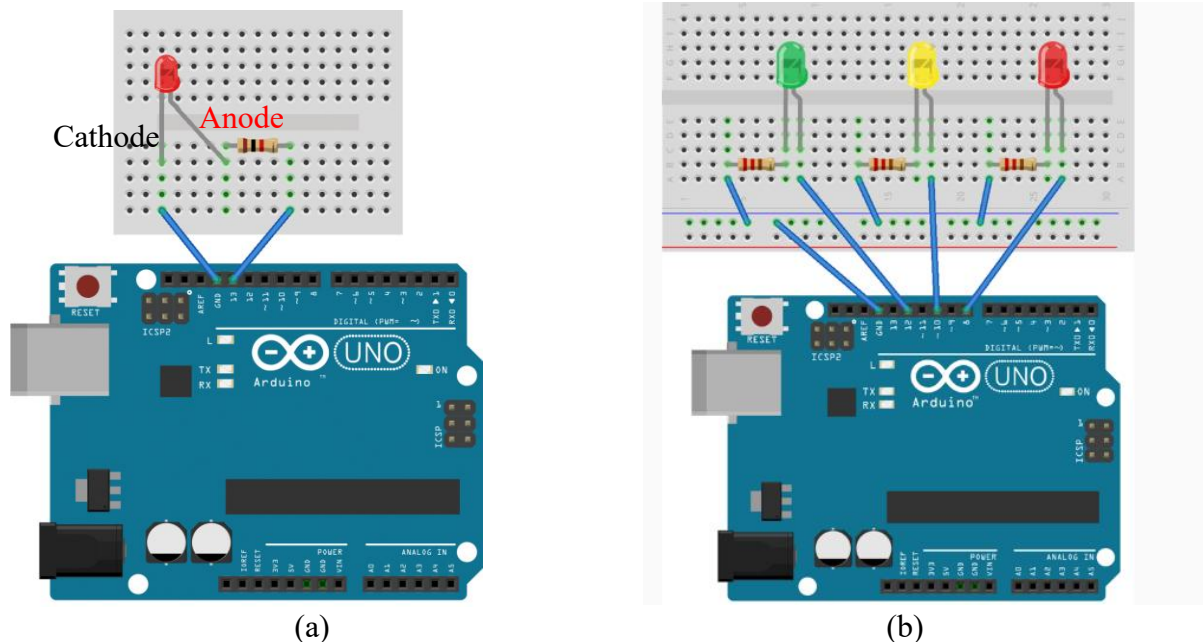


Fig. 3 Circuit connections for (a) LED blink test and (b) Traffic light control system

Program for Traffic Light Control System:

```
void setup() {
    // pin connections for the LED lights

    pinMode(8, OUTPUT);
    pinMode(10, OUTPUT);
    pinMode(12, OUTPUT);
}

void loop() {
    // Turning on the voltage at the output pin 8 (for green LED)
    digitalWrite(8, HIGH);
    delay(3000); // green LED is on for 3 seconds
    // Turning off the voltage at output pin 8 (for green LED)
    digitalWrite(8, LOW); // green LED is off

    // Turning the yellow LED on and off 4 times
    for (int i = 0; i < 4; i = i+1)
    {
        digitalWrite(10, HIGH);
        delay(500); // yellow LED is on for 0.5 seconds
```

```
digitalWrite(10, LOW);
delay(500); // yellow LED is off for 0.5 seconds
}

// Turning on the voltage at the output pin 12 (for red LED)
digitalWrite(12, HIGH);
delay(6000); // red LED is on for 6 seconds
// Turning off the voltage at the output pin 12 (for red LED)
digitalWrite(12, LOW); // red LED is off

} // void loop() function ends and repeats
```

3. After writing the program, you must save your code file in a folder. For this purpose, please go to the File Menu → Save As → give a File name in the file name box (for example, Traffic_light_control) → Select Save.

N.B. After saving your code, a sketch file with the desired file name will be stored in a folder with the same file name.

4. Now you need to verify/compile your code to find out and correct the errors. For this, please go to the Sketch Menu → Verify/Compile or press Ctrl+R or click on this button  just below the menu bar. If the code is correct then the code compilation done message will be displayed, otherwise, an error message will be displayed.

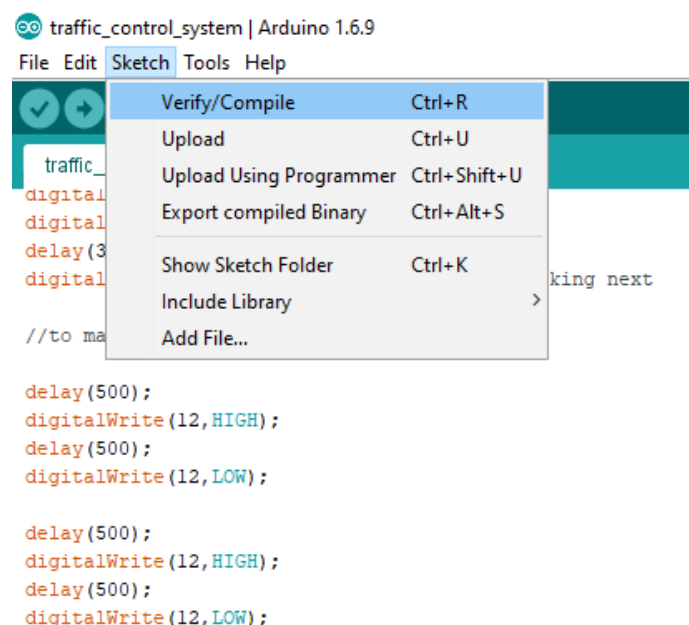


Fig. 4 Code compilation procedure.

5. After the compilation is done successfully, you need to upload your code onto the Arduino Uno board. To upload the program, connect your Arduino Uno R3 board to your PC with a USB cable. Before uploading the code, select the board type and port at your Arduino IDE. For this purpose, please go to the Tools Menu → Board: “Arduino Uno” → Arduino Uno. Then again go to the Tools Menu → Ports → COMx, here x means an integer number that corresponds to the Arduino Uno board that is automatically detected by the IDE.

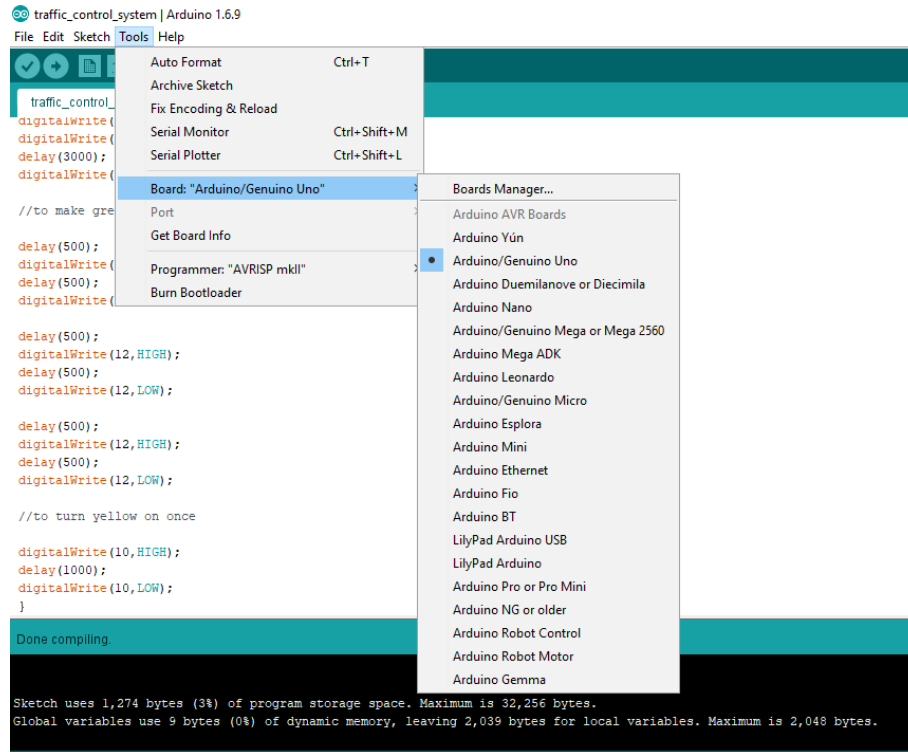


Fig. 5 Tools set up procedure.

6. After you have selected the board and port, you must upload the code. For this, please go to the Sketch Menu → Upload or press Ctrl+U or click on this button () just below the menu bar. If the code is correct, then the code upload-done message will be displayed; otherwise, an error message will be displayed. Based on the error message (if any), please go to the code and correct it. Usually, students forget to set up the Tools or select an inappropriate board or COMx port number to get such kinds of error messages.

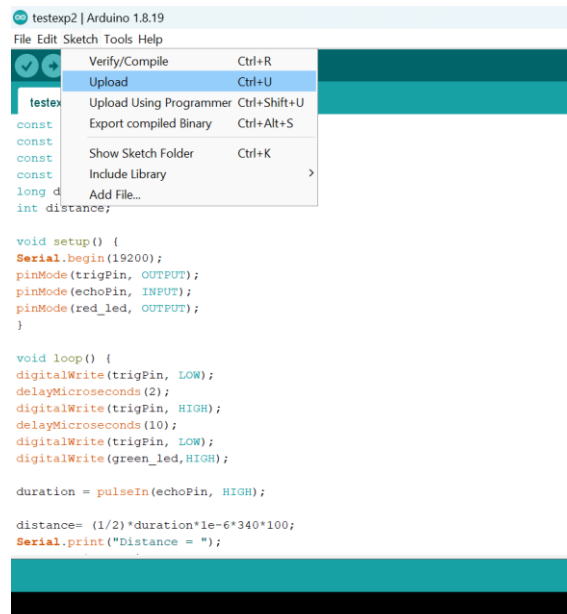


Fig. 6 Code uploading procedure.

How the Code Works:

Please explain the code with the flow charts.

Let us rewrite the code again for the same operation and same circuit by defining PINs and the duration of delays:

```
//PINs are defined by the corresponding LED colors
#define GREEN_PIN 8
#define YELLOW_PIN 10
#define RED_PIN 12

// The delay amounts are declared in ms; 1 s = 1000 ms
int red_on = 6000;
int green_on = 3000;
int yellow_blink = 500;

void setup() {
// pin connections for the LED lights
pinMode(GREEN_PIN, OUTPUT);
pinMode(YELLOW_PIN, OUTPUT);
pinMode(RED_PIN, OUTPUT);
}

void loop() {
//Turning on the voltage at output pin# 8 for the green LED
digitalWrite(GREEN_PIN, HIGH);
delay(green_on); // delays are in ms; 1 s = 1000 ms
//Turning off voltage at output pin# 8 for the green LED
digitalWrite(GREEN_PIN, LOW);

//Turning the yellow LED on and off 4 times
for(int i = 0; i < 4; i = i+1)
{
digitalWrite(YELLOW_PIN, HIGH);
delay(yellow_blink);
digitalWrite(YELLOW_PIN, LOW);
delay(yellow_blink);
}

//Turning on the voltage at pin# 12 for the red LED
digitalWrite(RED_PIN, HIGH);
delay(red_on);
//Turning off the voltage at output pin# 8 for the red LED
digitalWrite(RED_PIN, LOW);

} // void loop() function ends and repeats
```

Questions for report writing:

- 1) Include all codes and scripts in the lab report with detailed explanations according to the report writing template given in the portal.
- 2) Show the output/results in the form of images. Give their captions and descriptions.
- 3) Draw the program flow chart of both codes and explain them.
- 4) Include the **Proteus simulation** of the LED blink program and traffic light control system. Explain the simulation methodology. You may learn the simulation from the following video link: <https://www.youtube.com/watch?v=yHB5it0s2oU>

Reference(s):

- 1) <https://www.arduino.cc/>.
- 2) <https://www.coursera.org/learn/arduino/lecture/ei4ni/1-10-first-glance-at-a-program>.
- 3) Jeremy Blum; Exploring Arduino: Tools and Techniques for Engineering Wizardry.